1 ProSpin Tenis de Mesa

1.1 Informe de análisis del prototipo y propuesta de evolución tecnológica

Curso: DSY1104 – Desarrollo FullStack II

Documento base: Forma E – ProSpin Tenis de Mesa

Archivos analizados: `index.html`, `style.css` y `script.js`

Fecha: Septiembre de 2026

Resumen ejecutivo. El código entregado constituye un prototipo visual de catálogo para ProSpin Tenis de Mesa. La solución representa correctamente parte del contexto de la empresa, incorpora un catálogo de 80 referencias, filtros por categoría, un carrito y un formulario de registro. Sin embargo, todavía no corresponde a una aplicación FullStack, porque no utiliza React, no posee backend Spring Boot, no persiste información en una base de datos, no implementa autenticación con roles, no registra pedidos reales, no gestiona inventario ni integra mapas. Este informe diagnostica el estado actual, establece la brecha frente al documento académico y propone una arquitectura y un plan de implementación para convertir el prototipo en la solución solicitada.

2 1. Antecedentes y contexto del problema

ProSpin Tenis de Mesa es un negocio familiar ubicado en Santiago, Región Metropolitana, con doce años de trayectoria. Su catálogo incluye mesas de competición, maderas, gomas, pelotas, robots de entrenamiento, redes, vestuario y otros accesorios. Sus clientes son jugadores particulares, clubes, colegios y entrenadores que compran con distinta frecuencia y volumen.

El problema central identificado en el documento base es la dependencia de procesos manuales. La consulta de stock se realiza por teléfono y se verifica en bodega o en un archivo Excel. Las ventas se anotan en una libreta y el inventario se actualiza posteriormente. Las compras de instituciones con cuenta corriente se registran en cuadernos separados, lo que puede producir diferencias y disputas.

La solución requerida debe digitalizar la consulta de disponibilidad, el pedido, el inventario, las cuentas corrientes y la administración de usuarios. El sistema debe garantizar que el stock mostrado sea confiable, que los pedidos puedan rastrearse y que la información de cada cliente sea confidencial.

3 2. Objetivos del proyecto

3.1 Objetivo general

Diseñar la evolución del prototipo actual hacia una aplicación FullStack para ProSpin Tenis de Mesa que centralice el catálogo, el stock, los pedidos, el despacho, las cuentas corrientes y la administración de usuarios mediante una arquitectura segura, responsive y basada en servicios.

3.2 Objetivos específicos

1. Permitir que clubes, entrenadores y jugadores consulten el catálogo y el stock sin llamar a la tienda.
2. Implementar un flujo de cotización y pedido con carrito, retiro en tienda o despacho.

3. Mantener el inventario actualizado al registrar ventas físicas o digitales.
4. Incorporar alertas cuando un producto llegue a su punto de reposición.
5. Registrar el historial de pedidos y compras de cada cliente.
6. Digitalizar las cuentas corrientes de clubes y colegios.
7. Proteger la información mediante autenticación, autorización por roles y tokens JWT.
8. Integrar un mapa que muestre la tienda y la cobertura de despacho.

4 3. Tecnologías obligatorias y criterios de aceptación

El informe académico define una solución con React, Spring Boot, APIs REST en JSON, autenticación con roles, MySQL, Docker y AWS. Además, exige una interfaz responsive para anchos mínimos de 360, 768 y 1280 píxeles.

Capa	Tecnología o requisito	Criterio de aceptación
Frontend	React SPA	La interfaz se compone mediante componentes React y navegación sin recarga completa.
Backend	Spring Boot	Los servicios se ejecutan de forma independiente y exponen APIs REST.
Comunicación	HTTP y JSON	El frontend no accede directamente a la base de datos.
Seguridad	JWT y RBAC	Cada endpoint protegido valida el token y el rol del usuario.
Base de datos	MySQL, PostgreSQL u Oracle	El modelo se encuentra normalizado, como mínimo, en tercera forma normal.
Pedidos	Carrito, stock, retiro y despacho	El sistema registra el pedido y su estado, y valida disponibilidad.
Administración	Módulo React restringido	El administrador puede crear, editar, desactivar y asignar roles.
Mapas	Leaflet o Google Maps	Se muestra la ubicación de la tienda y el radio de cobertura.

Capa	Tecnología o requisito	Criterio de aceptación
Despliegue	Docker y AWS	La solución puede empaquetarse y desplegarse en la cuenta cloud.
Responsive	360, 768 y 1280 px	Menús, formularios, tablas y tarjetas se adaptan a los tres rangos.

5 4. Diagnóstico del código entregado

5.1 Estructura actual

El prototipo está compuesto por tres archivos principales. `index.html` contiene la estructura de la página y los modales. `style.css` contiene la identidad visual, las tarjetas de productos, el carrito y una regla básica para pantallas menores a 768 píxeles. `script.js` contiene el arreglo de productos, el renderizado del catálogo, los filtros, el carrito y la validación local del registro.

La separación por archivos es apropiada para una maqueta inicial. No obstante, la lógica se concentra en un único archivo JavaScript de más de mil líneas, lo que dificulta su mantenimiento y la reutilización de componentes.

5.2 Funcionalidades existentes

El prototipo presenta una sección institucional coherente con el contexto de ProSpin. También incluye un menú lateral con categorías de maderas, gomas, mesas, pelotas, pegamentos, limpiadores, vestuario y redes.

El archivo `script.js` contiene 80 productos identificados desde `TT-001` hasta `TT-080`. Cada producto posee identificador, categoría, marca, nombre, precio, descripción, etiqueta e imagen. El catálogo puede filtrarse por categoría y el usuario puede agregar productos al carrito.

El carrito calcula cantidades y total. El registro institucional valida nombre, correo, tipo de cliente, contraseña y confirmación de contraseña. Estos elementos constituyen una base visual útil para la futura migración a React.

5.3 Tabla de cumplimiento

Requisito	Estado	Evidencia o brecha
Catálogo de productos	Parcialmente cumplido	Hay 80 referencias en un arreglo local.
Filtros	Parcialmente cumplido	Funcionan en el navegador, sin persistencia ni búsqueda avanzada.
Carrito	Parcialmente cumplido	Calcula cantidades y total, pero no crea pedidos reales.

Requisito	Estado	Evidencia o brecha
Stock en tiempo real	No cumplido	El stock aparece dentro de textos y no se descuenta.
Registro	Parcialmente cumplido	Solo valida localmente y termina con un mensaje.
Inicio de sesión	No cumplido	El botón “Mi Cuenta” no implementa autenticación.
Roles RBAC	No cumplido	No existen permisos diferenciados.
JWT	No cumplido	No se envían tokens al backend.
API REST	No cumplido	No existe backend ni comunicación HTTP.
Spring Boot	No cumplido	No existen microservicios Java.
MySQL	No cumplido	El catálogo está embebido en JavaScript.
Pedidos	No cumplido	El checkout solo muestra una notificación y vacía el carrito.
Inventario	No cumplido	No hay entradas, salidas, alertas ni historial.
Cuenta corriente	No cumplido	Se menciona en el formulario, pero no se registra.
Panel administrativo	No cumplido	No existe una vista de administración.
Mapa	No cumplido	No hay integración con Leaflet ni Google Maps.
Responsive	Parcialmente cumplido	Existe un breakpoint, pero falta menú hamburguesa y validación en 360 px.
Docker y AWS	No cumplido	No hay configuración de despliegue.

6 5. Hallazgos técnicos relevantes

6.1 Datos de producto mezclados con la presentación

El stock está incluido dentro del campo descriptivo. Por ejemplo, el texto de cada producto contiene una frase como “Stock disponible: 60 uds.”. Esta representación impide validar cantidades desde el sistema y dificulta las actualizaciones.

El modelo debe separar los atributos de presentación y operación. Como mínimo, cada producto debería considerar `id`, `name`, `brand`, `category`, `subcategory`, `price`, `stock`, `minimumStock`, `color`, `thickness` y `size` cuando corresponda.

6.2 Carrito sin control de disponibilidad

La función que agrega productos incrementa la cantidad localmente, pero no compara la cantidad solicitada con el stock disponible. Tampoco reserva existencias durante el proceso de compra. En consecuencia, dos clientes podrían confirmar unidades que ya no existen.

El backend debe validar el stock dentro de una transacción. La confirmación debe fallar si la cantidad disponible es inferior a la solicitada. Después de crear el pedido, el sistema debe registrar el movimiento de inventario.

6.3 Checkout simulado

El botón “Confirmar Pedido” muestra una notificación de éxito, vacía el carrito y cierra el panel. No envía productos, cantidades, total, cliente, modalidad de entrega ni forma de pago a un servidor.

El flujo final debe crear un pedido con número único, fecha, estado, cliente, productos, cantidades, total, modalidad de entrega y condición de pago. Para clubes con cuenta corriente, el pedido debe registrar el estado “pago pendiente a fin de mes”.

6.4 Registro sin persistencia ni seguridad

El formulario de registro ejecuta validaciones en el navegador y muestra un `alert`. La contraseña no se envía a un servidor, no se cifra y no se crea un usuario real.

El sistema final debe aplicar hashing seguro en el backend, evitar almacenar contraseñas en texto plano y emitir un JWT después del inicio de sesión. Las rutas protegidas deben rechazar peticiones sin un token válido.

6.5 Responsive incompleto

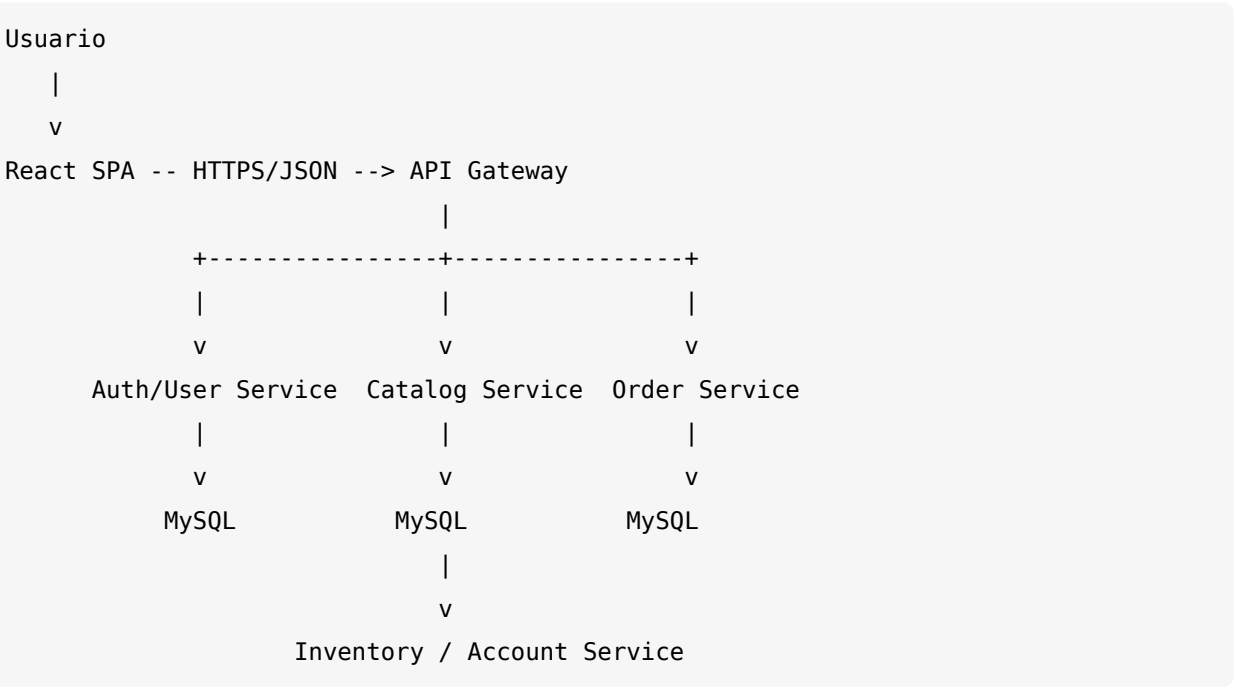
El CSS adapta el layout para anchos inferiores a 768 píxeles, pero el menú lateral no se convierte en un menú hamburguesa. Además, no existe una estrategia específica para el ancho mínimo de 360 píxeles. El carrito utiliza un ancho fijo que puede ser excesivo en pantallas pequeñas.

La solución debe utilizar un enfoque mobile-first. En 360 píxeles se debe priorizar una sola columna, botones táctiles, menú colapsable y formularios sin desbordamiento. En 768 píxeles se puede utilizar un panel lateral. En 1280 píxeles se pueden mostrar múltiples columnas.

7 6. Arquitectura propuesta

7.1 Vista general

La arquitectura recomendada separa la interfaz, los servicios de negocio y la persistencia. React se comunica únicamente con APIs REST. Los microservicios se comunican entre sí mediante HTTP interno y cada servicio mantiene una responsabilidad definida.



7.2 Microservicios sugeridos

Microservicio	Responsabilidad	Endpoints principales
Auth/User Service	Login, registro, usuarios, roles y JWT	`/api/auth/login`, `/api/usuarios`, `/api/roles`
Catalog Service	Productos, categorías, variantes y precios	`/api/productos`, `/api/categorias`
Inventory Service	Stock, movimientos y alertas de reposición	`/api/inventario`, `/api/movimientos`
Order Service	Carrito confirmado, pedidos y estados	`/api/pedidos`, `/api/pedidos/{id}/estado`
Account Service	Cuenta corriente, saldos y pagos	`/api/cuentas`, `/api/cuentas/{id}/movimientos`
Delivery Service	Cobertura, direcciones y modalidades de entrega	`/api/cobertura`, `/api/despachos`

7.3 Roles y permisos

El rol **Administrador** puede gestionar usuarios, asignar roles, consultar reportes y visualizar toda la operación. El rol **Vendedor o Empleado** puede administrar stock y actualizar el estado de pedidos, pero no puede modificar usuarios. El rol **Club, Entrenador o Cliente registrado** puede consultar el catálogo, crear pedidos, consultar su historial y visualizar su saldo si posee cuenta corriente.

La autorización debe implementarse en dos niveles. El frontend debe ocultar vistas no permitidas para mejorar la experiencia. El backend debe imponer la regla definitiva mediante anotaciones o filtros de Spring Security, porque ocultar un botón no es una medida de seguridad.

8 7. Modelo de datos propuesto

El modelo debe mantener entidades separadas y relaciones normalizadas. Las variantes de gomas deben almacenar color y grosor como atributos propios. Las tallas de vestuario y calzado también deben estar separadas de la descripción libre.

Entidad	Campos relevantes	Relación
`usuarios`	id, nombre, email, password_hash, estado	Tiene un rol y puede generar pedidos.
`roles`	id, nombre	Se relaciona con usuarios.
`productos`	id, sku, nombre, marca, categoría, precio, stock, stock_minimo	Puede tener variantes.
`variantes_producto`	id, producto_id, color, grosor, talla	Permite identificar la especificación comprada.
`pedidos`	id, numero, usuario_id, estado, total, entrega, fecha	Contiene varios ítems.
`detalle_pedido`	pedido_id, variante_id, cantidad, precio_unitario	Representa los productos del pedido.
`movimientos_inventario`	id, variante_id, tipo, cantidad, usuario_id, fecha	Registra entradas, ventas y ajustes.
`cuentas_corrientes`	id, cliente_id, limite, saldo, estado	Una institución puede tener una cuenta.
`movimientos_cuenta`	cuenta_id, pedido_id, tipo, monto, fecha	Registra cargos y pagos.
`zonas_despacho`	id, centro_latitud, centro_longitud, radio_km	Define la cobertura de entrega.

9 8. Flujos funcionales principales

9.1 Consulta y pedido

El cliente inicia sesión y consulta el catálogo. Puede filtrar por marca, categoría, color, grosor, talla y disponibilidad. Al agregar productos, el frontend solicita o recibe el stock actualizado. Al confirmar, el backend vuelve a validar la existencia y crea el pedido dentro de una transacción.

El cliente elige retiro en tienda o despacho. En el segundo caso, el sistema verifica que la dirección se encuentre dentro de la zona definida. El pedido inicia en estado “pendiente” y puede avanzar a “confirmado”, “en preparación”, “listo”, “enviado” y “completado”.

9.2 Inventario

Cada venta digital o física debe generar un movimiento de salida. Cada recepción de mercadería debe generar un movimiento de entrada. Los ajustes deben exigir un usuario responsable y una observación. Cuando el stock sea menor o igual al mínimo, el sistema debe crear una alerta para el dueño o administrador.

9.3 Cuenta corriente

El sistema debe permitir asociar una cuenta corriente a un club, colegio o institución aprobada. Cuando se confirme una compra a fin de mes, el pedido debe aumentar el saldo pendiente. El cliente debe ver sus movimientos y el administrador debe consultar el total adeudado, los vencimientos y el historial de pagos.

10 9. Diseño de interfaz recomendado

La estética actual utiliza una combinación de azul oscuro, rojo y tonos neutros. Esta dirección visual es coherente con una tienda especializada y puede mantenerse durante la migración. La interfaz React debe transformar las secciones actuales en componentes reutilizables.

Componente	Responsabilidad
`Navbar`	Marca, sesión, carrito y menú móvil.
`Sidebar`	Categorías y navegación según rol.
`ProductGrid`	Presentación responsive de productos.
`ProductCard`	Imagen, precio, stock, variantes y acción de compra.
`CartDrawer`	Productos seleccionados, cantidades, entrega y total.
`AuthModal`	Inicio de sesión y solicitud de registro.
`OrderHistory`	Pedidos del cliente y estados.
`InventoryTable`	Stock, movimientos y alertas para empleados.
`AdminUserTable`	Gestión de usuarios y roles.

Componente	Responsabilidad
`CoverageMap`	Tienda, radio de despacho y validación de dirección.

El menú lateral debe convertirse en un menú hamburguesa en móviles. Las tablas de inventario deben permitir desplazamiento horizontal o transformarse en tarjetas. Los formularios deben usar mensajes de error accesibles y mantener el foco visible.

11 10. Plan de implementación

11.1 Fase 1: Migración del prototipo a React

Se deben crear componentes para la navegación, contexto institucional, catálogo, tarjeta de producto, carrito y formularios. El arreglo local de productos puede utilizarse temporalmente como fuente de datos para validar la interfaz, pero debe quedar preparado para ser reemplazado por una API.

11.2 Fase 2: Backend y persistencia

Se deben crear los proyectos Spring Boot y las bases de datos correspondientes. Después se implementan los repositorios, servicios, controladores REST y respuestas JSON de error. El modelo debe validarse con datos reales del catálogo.

11.3 Fase 3: Seguridad y roles

Se implementan registro, login, hashing de contraseñas, emisión de JWT, validación del token y autorización por rol. Se deben probar accesos permitidos y rechazados para cada endpoint.

11.4 Fase 4: Pedidos e inventario

Se implementan carrito, validación de stock, creación de pedidos, estados, reservas o descuentos de inventario, alertas y registro de movimientos. Esta fase constituye el núcleo operativo del proyecto.

11.5 Fase 5: Cuenta corriente y cobertura

Se implementan cuentas institucionales, saldos, cargos, pagos y el mapa. La validación de cobertura debe ejecutarse antes de permitir la modalidad de despacho.

11.6 Fase 6: Docker, pruebas y AWS

Cada microservicio debe incorporar su `Dockerfile`. El entorno local debe contar con un archivo `docker-compose.yml`. Finalmente se deben desplegar los servicios en AWS y probar la aplicación en los tres tamaños requeridos.

12 11. Pruebas de aceptación

Caso	Resultado esperado
Cliente consulta una goma por color y grosor	Solo aparecen variantes compatibles y su stock actual.

Caso	Resultado esperado
Dos usuarios solicitan la última unidad	Una confirmación es exitosa y la otra recibe stock insuficiente.
Cliente sin token accede a pedidos	El backend responde con error de autorización.
Vendedor intenta gestionar usuarios	La operación es rechazada por falta de rol.
Administrador desactiva un usuario	El usuario no puede iniciar nuevas sesiones.
Club compra con cuenta corriente	Se registra el pedido y aumenta el saldo pendiente.
Producto llega a stock mínimo	El administrador recibe una alerta de reposición.
Dirección está fuera del radio	El sistema no permite seleccionar despacho.
Vista en 360 píxeles	Menú colapsado y contenido sin desbordamiento horizontal.
Pedido cambia a “listo”	El cliente visualiza el nuevo estado en su historial.

13 12. Conclusiones

El código entregado es una base visual útil y representa correctamente varios elementos del negocio descrito en el informe. La presencia de 80 productos, categorías, filtros, carrito y registro permite reutilizar una parte importante de la presentación inicial.

La brecha principal se encuentra en la ausencia de persistencia, backend, seguridad y procesos de negocio. El sistema actual no puede garantizar stock real, crear pedidos verificables, controlar cuentas corrientes ni restringir la información según el usuario.

La recomendación es conservar la identidad visual y migrar progresivamente la interfaz hacia React. La implementación debe priorizar autenticación, catálogo persistente, inventario y pedidos. Luego deben incorporarse cuentas corrientes, administración, mapas, Docker y despliegue en AWS. Con este orden se reduce el riesgo técnico y se ataca primero el problema operativo más importante: la disponibilidad confiable de productos y el registro correcto de las ventas.

14 Referencias

[8]: “Documento de contexto DSY1104 – ProSpin Tenis de Mesa, Forma E, entregado por el usuario.”

[9]: “Prototipo entregado por el usuario: index.html, style.css y script.js.”